여러 repository의 문서를 memory-mcp로 만들어 코딩 에이전트에게 먹이기

meta-repository → 문서 생명주기 → OpenSearch memory-mcp → knowledge graph

Jin · 당근마켓 광고실 데이터 플랫폼팀

"당신의 지능을 소유하라"

— 개리 탠의 인용. 출처: 와이콤비네이터 스타트업 스쿨 2026 키노트

모델은 빌리는 것,
컨텍스트는 소유하는 것

모델이 좋아질수록
차별점은 컨텍스트로 옮겨간다

모델 업그레이드는
내 라이브러리에 공짜로 붙는다

당신이 만약, 3개월 전 입사했는데, 지난 1년의 결정 중에, 왜 그렇게 했는지를 물어본다면 할 수 있는 말은?

저는 그 때 없었는데요?

이유가 있었겠죠? ㅎㅎㅎㅎ

혹시 그 팀의 문서는 슬랙 메세지 인가용?

전화번호 기억하시나요?

history 또한
전화번호가 되어야한다 생각해요.

와이프 전화번호는 당연히 외우겠죠 ㅎㅎ

하지만 우리 팀리더 전화번호는 몰라도 되잖아요 ㅎㅎ

"010 으로 시작하는건 알아요 ㅎㅎ"

질문 하나

코딩 에이전트가 만든 개발문서, 다들 어떻게 관리하고 계신가요?

01

meta-repository를 운영한다

02

repository마다 docs에 넣어 둔다

03

문서는 찌꺼기다

SSoT는 언제나 소스코드라는 철학

매일 겪던 일

매 세션마다
"이 프로젝트는 이런 배경이고,
이 용어는 이런 뜻이고,
지난번엔 이렇게 결정했고..."

문제

광고실 데이터 플랫폼팀이 하는 일은 두 문장

광고 어트리뷰션을 수집해서 제공한다

활성 광고 데이터를 만들어 서빙 파트에 넘긴다

비즈니스 로직이 집중된 곳. 사이드 이펙트 추론이 어렵다

스키마가 서비스 레이어와 결합 → repository 하나로는 사이드 이펙트를 볼 수 없다

문제

그래서 매일 이런 상황이었습니다

1 한 줄도 기여하지 않은 repository의 코드를 반드시 읽어야 한다

2 다른 팀 repository가 어느 방향으로 움직이는지도 알고 있어야 한다

3 그 사실을 에이전트에게 매 세션 처음부터 다시 설명해야 한다

설명을 빼먹으면 없는 history를 지어낸다. 고치다 보면 하루가 간다.

얻고 싶었던 것은 딱 하나

AI가 알아서 그 repository의 history를 이해했으면 좋겠다

- 매 세션 지난 history를 설명하지 않아도, 없는 history를 지어내지 말 것
- 특정 repository 작업이 시작되면 알아서 지난 history를 읽어 갈 것

해법

한 번에 만든 게 아닙니다. 문제가 드러날 때마다 한 단계씩 올렸습니다

1단계

meta-repository와 디렉토리 구조

계기

매 세션 history를 다시 설명해야 했다

결과

INDEX.md만으로, OpenSearch 없이 운영

2단계

바깥 문서 불러오기와 생명주기

계기

문서는 밖에도 있었고, 쌓인 문서는 낡아 토큰을 태웠다

결과

60일 compaction, 원문은 S3로

3단계

OpenSearch memory-mcp

계기

meta-repository가 너무 커져 탐색이 오래 걸렸다

결과

hot만 색인, cold는 S3에서 다시 적재

4단계

knowledge graph DB

계기

사실과 유효한 규칙을 늘 최신으로 유지해야 했다

결과

knowledge는 파일 대신 원격 저장소로

1단계 · meta-repository

meta-repository: repository들의 메타 정보를 모아 둔 repository

메타 정보 = 네 가지

spec

ADR

PRD

AI plan

에이전트가 작업 전에 세운 계획

1단계 · meta-repository

제가 제안한 디렉토리 구조

```
meta-repo/
├── CLAUDE.md
├── repos/
└── docs/
    ├── repository/{repo}/
    │   └── INDEX.md
    ├── feature/{yyyy-mm-dd}_{topic}/
    │   └── INDEX.md
    ├── specs/{yyyy-mm-dd}_{spec}/
    │   └── INDEX.md
    └── knowledge/{topic}/
        └── INDEX.md
```

repository/{repo}
그 repository의 ADR, 배경, 용어 매핑. 언제나 최신이라는 전제

feature/{yyyy-mm-dd}_{topic}
여러 repository에 걸친 기능. 날짜 디렉토리, 안은 episodic

specs/{yyyy-mm-dd}_{spec}
사람이 쓴 스펙, AI가 만든 스펙, 작업 plan. 날짜 디렉토리, 안은 episodic

knowledge/{topic}
주제별 사실과 규칙만. 날짜를 타지 않는다

INDEX.md는 사람에게 진입점, 에이전트에게는 전체 대신 먼저 읽는 목차. **이 덕분에 OpenSearch 없이도 운영할 수 있었다.**
검색엔진이 들어오면 INDEX.md 의 운영은 불필요해짐.

1단계 · 디렉토리마다 전제가 다르다

이 문서는 날짜를 타는가, 타지 않는가

repository/

언제나 최신

이 아래 문서는 항상 지금 상태라는 전제

feature/ · specs/

날짜를 탄다

{yyyy-mm-dd}_ 디렉토리. 안의 데이터는 거의 전부 episodic

knowledge/

날짜를 타지 않는다

주제 디렉토리. 사실과 규칙만. 생명주기를 타지 않는다

1단계 · 규칙

작업은 항상 새 worktree에서, 규칙은 CLAUDE.md에

worktree를 쓰는 이유 세 가지

1. 에이전트가 여러 작업을 동시에 해도 디렉토리가 분리되어 브랜치가 섞이지 않는다

2. 작업 경로가 고정되어 어떤 변경이 어느 작업에서 왔는지 추적된다

3. codebase-memory-mcp가 worktree를 감지해 코드 위치를 답해 준다

CLAUDE.md

· 작업 시작 시 그 repository의 INDEX.md부터 읽을 것 (3단계부터는 search_memory 먼저)

· 문서로 정보를 만들 때는 "추론 없이"

· 결과 문서는 어느 디렉토리에 어떤 frontmatter로 저장할 것

· **이 규칙들의 변경 내역.** 그래야 "예전 규칙으로 만들어진 문서"를 구분한다

1단계 · 스킬

티켓 하나만 넣으면 끝까지 가도록

1 Jira · Linear 티켓 입력	2 작업	3 문서 생성	4 티켓 상태 이동
5 Slack 알림	6 블랙박스 테스트 설계 → Linear 댓글	7 배포	8 e2e 테스트 자동 실행

스킬은 대강, 대충 만들었습니다. **AI한테 만들어 달라고 해서.** 중요한 건 3번, 작업이 끝나면 문서가 meta-repository에 남는다는 것.

2단계 · 다른 곳에서 문서를 불러온다

문서는 meta-repository 밖에도 있었습니다

기능의 추가 개발과 수정

Notion 문서 싱크

관리하는 repository의 모든 문서



`feature/`

날짜 디렉토리. episodic으로 쌓인다

`knowledge/`

여기로 바로 들어오지 않는다. 생명주기에서 살아남은 사실과 규칙만

2단계 · 문서의 생명주기

60일 지난 문서는 compaction, 원문은 S3로

1
대상
60일이 지난 episodic 문서(feature, specs). 문서 나이 기준

2
compaction
그 문서들로 새 문서를 만든다

3
반영
feature나 knowledge에 새 문서로, 또는 기존 문서에 append. S3 경로도 함께 남긴다

4
이동
원문은 S3로. hot → cold

5
PR
이 수정이 PR로 만들어진다

지우지 않는다. 원문은 cold로, 살아남은 내용은 hot으로. 이 단계의 hot/cold는 아직 OpenSearch가 아니라 **파일과 S3 경로**였다. **knowledge는 생명주기를 타지 않는다.**

여기서 compaction은 세그먼트 병합이 아니라 문서를 다시 써서 줄이는 일, cold는 데이터 티어가 아니라 S3 보관이다

3단계로 넘어간 이유: meta-repository가 너무 커져 탐색이 오래 걸렸다

비슷한 문서를 전부 열어 읽는 일은 검색엔진이 수십 년 해 온 IR 그 자체다

에이전트가 직접 하면

파일 트리 탐색, ag 검색 → 열기 → 읽기 → 비교. 비슷한 문서마다 반복되어 시간과 토큰이 쌓인다

OpenSearch가 하면

BM25 질의 한 번. 에이전트는 상위 청크만 받는다

3단계 · 요구사항

OpenSearch 기반 memory-mcp에 필요한 조건 네 개

목표는 episodic 문서 조회. 사실과 규칙들의 충돌 관리는 처음부터 포기.

#	요구사항	이유
1	GitHub·로컬의 문서 파일을 OpenSearch에 담을 것	문서는 이미 meta-repository에 파일로 있음
2	팀/실 단위로 문서를 분리 관리할 것	광고실 안에서도 팀마다 문맥이 다름
3	hot / cold 영역이 있을 것	1년, 2년 지난 문맥을 참조할 일이 생길 수 있음. 오래된 history도 오래 남기고 싶었다
4	cold는 S3에 적재하고, 필요하면 OpenSearch로 바로 로드할 것	지우긴 아깝고, 항상 켜 두긴 비쌈

3단계 · 구현

구현하기로 한 기능 여섯 가지

1 OpenSearch는 우선 **로컬 머신**에 올린다. 팀 공용 클러스터는 나중 문제

2 **일단 전부 넣는다.** 아무리 못 만들어도 ag 검색보다는 낫다는 생각으로

3 **파일 워칭.** docs 아래의 모든 수정은 수정 색인 대상

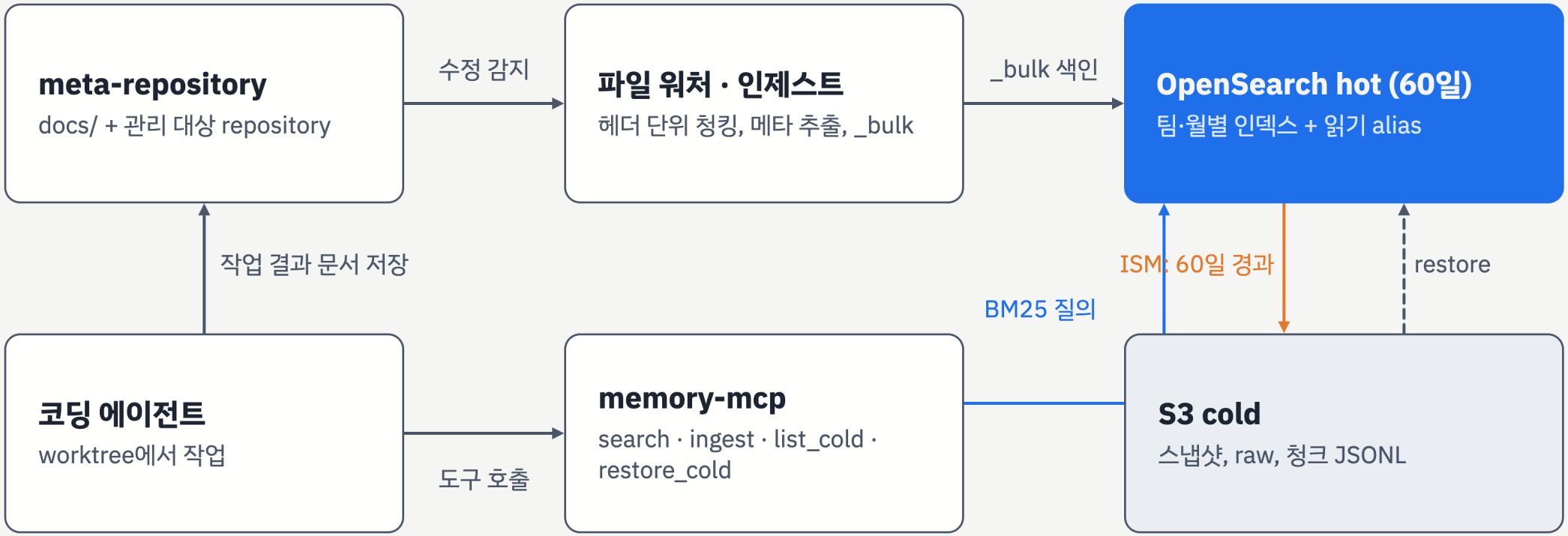
4 처음에는 임베딩 없이 **BM25만**. 비용 문제. remote 모드로 올릴 때 임베딩 추가

5 **hot만** OpenSearch에 적재. 기준은 2단계의 60일 그대로

6 **cold가 필요하면** S3에서 내려받아 OpenSearch에 다시 적재

3단계 · 구현

에이전트는 파일 트리 대신 OpenSearch에 묻는다



3단계 · 짚고 갈 것

hot/cold는 로그 운영 패턴을 그대로 가져왔습니다

episodic 문서는 추가만 되고 시간순으로 쌓인다. 로그 데이터와 성질이 같다.

레이어	구성
인덱스	mem-{team}-{yyyy.MM} 월별 인덱스. 읽기 alias가 hot 인덱스 전체를 가리킴. 팀 단위 분리는 인덱스 이름과 alias로 끝남
문서 필드	팀·repository·문서 종류·시간·경로 메타 + 본문. 시간 범위 필터가 episodic 조회의 핵심
분석기	인덱스 기본 분석기를 nori(한국어 형태소 분석)로
청킹	마크다운 헤더 계층 단위. 코드 블록 안에서는 자르지 않음
ISM 정책	hot 60일 → 스냅샷으로 S3 repository 저장 → 인덱스 삭제. 필요하면 restore로 인덱스 단위 재로드. knowledge 문서는 대상이 아니다
S3 층	원본 raw, 파싱된 청크 JSONL, 스냅샷. 매핑 변경이나 임베딩 추가 시 JSONL에서 _bulk로 재적재

3단계 · 구현

MCP 도구는 네 개면 충분했습니다

`search_memory(team, query, time_range)`

hot alias에 BM25 검색. 결과는 source_path와 header_path를 포함한 청크

`ingest_path(path, team)`

워처가 놓친 경우를 위한 수동 적재

`list_cold(team)`

S3에 있는 cold 인덱스 목록

`restore_cold(index)`

스냅샷 restore. cold 범위 질의가 오면 에이전트가 "복원할까요"를 먼저 묻는다

3단계 · 데모

배경 설명 없이 작업을 시켜 보겠습니다

1. meta-repository의 docs 아래에 "활성 광고" 용어 매핑을 담은 ADR 하나를 저장한다. 워처가 잡아서 색인되는 로그를 본다
2. 에이전트 세션을 새로 열고, "활성 광고"와 관련된 작업을 시킨다. 배경 설명 없이
3. 에이전트가 search_memory를 부르고, 돌아온 청크의 source_path를 근거로 작업하는 것을 본다
4. 같은 작업을 memory-mcp 없이 돌린 화면과 나란히 본다

3단계 · 측정

숫자는 없습니다.
써 보면 압니다. (아마도..?)

파일 트리를 뒤지고 같은 문서를 몇 번씩 열던 시간과 토큰이
눈에 띄게 줄었습니다.

4단계로 넘어간 이유

episodic은 충돌을 무시해도 되고, knowledge는 항상 신경 써야 합니다

구분	episodic 문서	knowledge 문서
정체	시간순으로 쌓이는 작업 기록. ADR, plan, 회의 결정	걸러져 살아남은 사실. 용어 정의, 시스템 불변식
저장소와 IR	역색인 DB. 지금은 OpenSearch BM25	graph DB(neo4j). 파일이 아닌 원격 저장소
생명주기	반드시 있어야 함. 60일 hot, 이후 cold	영속. 대신 갱신 이력이 필요
사실 충돌	무관해도 됨. 같은 주제에 다른 기록이 공존하는 게 정상	항상 주의. A/B 중 택일하거나 C를 만들어야 함
흐름	compaction 때 살아남은 사실이 knowledge로 넘어감	episodic에서만 들어옴

4단계 · knowledge graph DB (neo4j)

사실과 유효한 규칙을 늘 최신으로 유지하려고 knowledge를 graph DB로 옮겼다

들어오는 길

2단계 compaction에서 살아남은 사실과 규칙

해 보니

생각보다 이 정보는 파일보다 **원격 저장소**가 더 좋았다

Demo

https://github.com/drakejin/workspace

다른 곳의 사례 · 컨텍스트 엔지니어링

같은 문제를 Elastic은 "컨텍스트 엔진"으로 풀었습니다

"컨텍스트 엔진은 질문이 오기 전에 데이터를 이해한다"

— 아니시 매서의 인용. 출처: Elastic Search Labs, Context engineering for agents, 2026-10-08

Elastic이 꼽은 세 가지 문제 · 우리 경우

질의마다 드는 탐색 비용	1·3단계로 대응
컨텍스트 표류	2단계로 대응
고립된 지식	아직 · 로컬 OpenSearch

같은 세 지표 · 없이 → 있음

지표	Elastic 컨텍스트 엔진	우리 memory-mcp
도구 호출	33 → 9	비슷하겠지..?
입력 토큰	2,279,253 → 653,511 (71% 감소)	비슷하겠지..?
응답 시간	213초 → 64초	비슷하겠지..?

Elastic: 고객 데이터에 대한 질문 한 건, 같은 모델.
우리: 개발문서 작업. 따로 재 보지는 않았다.
데이터 영역이 달라 절대값이 아니라 방향을 비교한다

컨텍스트 레이어의 연속 루프에 이 발표를 대입하면

수집·구축·검색은 있다. 개선이 비어 있다

수집 → 구축 → 검색 → **개선** → 다시 수집. 사용 기록이 새 소스가 된다

단계	이 발표에서 대응한 것	비어 있는 것	상태
수집 소스를 읽고 찾는다	meta-repository, 관리 repository 문서, Notion 싱크, 기능 개발·수정. 모두 feature/로, 파일 워처가 잡는다	Slack 대화와 티켓 댓글은 소스로 들어오지 않는다	대응
구축 지식 단위로 저장한다	INDEX.md, frontmatter·header_path 메타. 60일 compaction으로 사실·규칙을 knowledge로, 원문 S3 경로 기록	지식 단위가 사실·규칙 두 가지뿐. 데이터의 함정, 질의 요령 같은 단위는 없다	대응
검색 관련 단위만 작게 가져온다	search_memory가 hot alias에 BM25(nori)로 상위 청크만 준다. cold는 묻고 나서 복원	의미 검색 없음. 도구 넷이 모두 OpenSearch 대상이라 knowledge graph까지 한 번에 못 묻는다	부분
개선 사용 기록이 다시 소스로	CLAUDE.md 규칙의 변경 내역. compaction PR을 사람이 검수	사용 기록을 모으지 않는다. 검색 실패나 search_memory 미호출이 다시 들어오지 않는다	미비

"에이전트의 컨텍스트 레이어를 설계한다면, 검색 위에 지어라" — 아니시 매서의 인용. 출처: Elastic Search Labs, Context engineering for agents, 2026-10-08

빠진 고리 · 개선

지금은 한 방향 파이프라인.
쓰임이 다시 들어와야 루프가 된다

1

사용 기록을 소스로

search_memory의 질의·결과·도구 호출을 UBI로 남기고 compaction 때 함께 본다

2

검색 품질을 잰다

질의 세트와 판정 목록으로 BM25(nori) 결과를 재고, 바꿀 때마다 실험으로 비교한다

3

팀이 함께 쓰는 클러스터

로컬 OpenSearch를 팀 공용으로. 한 사람의 에이전트가 배운 것을 팀이 쓴다

4

한 번에 묻기

episodic과 knowledge를 한 질의로. BM25에 의미 검색을 더해서

재료는 OpenSearch에 이미 있다. **사용자 행동 기록(UBI)**과 **검색 적합성 워크벤치**(질의 세트·판정 목록·실험). 공식 OpenSearch MCP 서버에도 도구로 들어 있다

마무리

정리

memory-mcp 하나로 끝나지 않습니다. 같이 쓰면 좋았던 세 가지

codebase-memory-mcp

코드 구조를 그래프로 들고 있어 "이 필드를 만드는 함수가 어디냐"에 파일 탐색 없이 답한다. 문서는 OpenSearch, 코드는 이것

주석 기반 코드 history

괴이한 history가 섞인 멤버 변수에는 왜 그런지를 주석으로. 코드를 읽는 순간 함께 읽히는 가장 가까운 컨텍스트

episodic과 knowledge의 분리

episodic: 시간순으로 쌓이는 작업 기록. 생명주기가 있고, 같은 주제에 다른 기록이 공존해도 된다

knowledge: 걸러져 살아남은 사실. 영속하고, 충돌하면 하나로 정리해야 한다

정리

말하고 싶은 것 다섯 가지

1 **meta-repository부터 운영하세요.** 디렉토리 구조와 INDEX.md만으로 시작할 수 있습니다. 흩어진 문서는 에이전트가 찾지 못하고, 찾지 못하면 지어냅니다

2 **문서에 생명주기를 주세요.** 그래야 문서가 찌꺼기가 되지 않습니다. 계속 가공하고, 압축하고, 최신화하세요. 문서가 커지면 IR은 OpenSearch에 맡기세요

3 **코드베이스에는 codebase-memory-mcp를 로컬에서 돌리세요.** 매 세션의 파일 탐색 토큰과 시간을 아낍니다

4 **메모리 설계가 에이전트의 성능을 가릅니다.** ["인용 문장" — 누구의 인용. 출처: 형식으로 두 건 채울 것]

5 **knowledge는 graph DB로 운영하세요.** 사실과 규칙은 파일보다 원격 저장소가 더 잘 맞았습니다

knowledge를 운영하면 이런 고민을 하게 됩니다

episodic에서 넘어온 사실이 기존 사실과 부딪히면, A인가 B인가 C인가를 누가 정하는가

지금 생각하는 답

충돌하면 **최신이 이긴다.** 최신이 거짓 같아 보이면 사람이 검수한다

아직 모르는 것

"거짓 같아 보인다"를 누가, 무엇으로 판단하는가

여러분은 어떻게 하실 건가요

정답을 드리러 온 게 아니라, 같이 이야기하러 왔습니다

감사합니다

질문과 이야기를 받겠습니다

Jin · 당근마켓 광고실 데이터 플랫폼팀

https://github.com/drakejin/workspace

예상 질의응답 · 참고 자료

받을 법한 질문과 답

Q공식 OpenSearch MCP 서버는 왜 안 썼나요?

A팀·조직 개념을 넣고 싶었고, memory mcp 에 대해 잘 몰랐고, 배우려는 목적도 있었습니다.

Q그 규모면 ripgrep으로 충분하지 않나요?

Aripgrep으로 시작했습니다. 어느 디렉토리를 볼 것인가를 찾고 있는걸 보면 답답합니다. 찾고 다시 읽는 반복이 토큰을 더 썼고, 같은 단어가 든 문서들 사이의 뉘앙스는 OpenSearch가 낫다는 직감이었습니다.

Qknowledge는 왜 OpenSearch가 아니라 neo4j인가요?

AOpenSearch에는 역색인 저장소 역할만 주고, 그 이상은 맡기지 않는 게 마음이 편했습니다.

Qcompaction 요약이 정보를 빠뜨리거나 지어내면요?

A검증하지 못했습니다. 이번 데모의 맹점입니다. 어떤 방법이 있을까요. coding agent 가 알아서 만족도를 memory-mcp 에 전달하여 ubi 플러그인을 쓴다던가..?

Q같은 주제의 기록이 서로 부딪히면요?

Aepisodic에서는 공존이 정상이고 최신을 우선합니다. knowledge로 넘길 때만 사람이 판단하고, 자동화는 아직입니다.

Q에이전트가 search_memory를 안 부르면요?

ACLAUDE.md에 작업 시작 규칙으로 적어 둡니다. 그래도 빠지면 세션 시작 훅에서 자동 검색하는 방식을 실험 중입니다.

영상	Garry Tan: Own Your Intelligence	youtube.com/watch?v=eRrc1pUY5oU
블로그	Context engineering for agents · Elastic Search Labs	elastic.co/search-labs/blog/context-engineering-for-agents
저장소	OpenSearch 공식 MCP 서버	github.com/opensearch-project/opensearch-mcp-server-py